

一，什么是数据处理引擎？

Jiutian 数据处理引擎是面向小艺多模态业务打造的一站式数据加工平台。原始多模态数据经由引擎清洗拆解，提炼出语义、视觉、声学等各类特征向量；经过规整后字段统一、维度标准化，完全匹配小艺大模型的入参规范，从而开展推理与训练。

二、Jiutian 数据处理引擎完整工作流程

Jiutian数据处理引擎以标准化API串联**数据输入**、**数据转换**、**数据输出**全流程，整体分为三步：

第一步：数据输入

引擎支持三类数据源接入，配套专属读取接口：

1. **持久化存储数据**：磁盘、OBS、数据库等存量文件数据，通过 `read_videos`、`read_parquet` 等接口读取；
2. **Python 内存数据**：Python字典列表等内存对象，通过 `from_items` 快速生成 DataFrame 载入；
3. **自定义数据源**：业务特有非标数据源，依托用户自定义函数，进行数据读取。

第二步：数据转换

依托内置算子完成数据清洗、格式转换、特征提取，分为两类处理逻辑，**每一步算子均可单独配置 CPU/NPU 算力、Python 运行环境及三方依赖**：

1. **映射类运算**：`map` 实现字段一对一转换、`flat_map` 实现一条数据拆解为多条数据，搭配 `groupby` 完成数据分组聚合；
2. **过滤类运算**：通过 `filter` 剔除无效、冗余、异常数据，完成数据清洗。

经过多轮算子处理后，原始多模态数据被加工为规范的特征向量。

第三步：数据输出

加工完成的数据提供三种输出方式：

1. **落地持久化存储**：通过 `to_json`、`to_parquet` 等接口存入本地文件、或者s3，obs；
2. **导出至 Python 内存**：借助 `iter_rows` 生成数据迭代器，数据直接留在内存供小艺模型实时调用；

3. **自定义落地规则**：通过用户自定义函数，对接个性化存储目标。

三，为什么原始数据不能直接传给大模型

一是原始数据可能涉及隐私内容，存在合规泄露风险；二是脏数据多，容易引发模型幻觉出错；三是超 Token 长度限制、浪费资源；四是缺少结构化释义，模型无法识别业务含义，因此必须清洗、切片后再接入大模型。

四，介绍比较印象深刻的缺陷

1, delete_job 删除作业成功，但重新提交同名 ID 作业失败

我之前在测作业管理模块时碰到过一个很典型的bug。这个模块有两个接口函数，一个是提交作业 submit_job，一个是删除作业delete_job。每个作业都有的唯一编号job_id，需求就是作业没删除掉的话，不能重复提交相同job_id的任务。

当时测试步骤是：等作业跑完，调用delete_job删除作业，前端也提示删除成功了。可我紧接着用原来的 job_id 重新提交作业，结果提交报错了。

后面跟定位发现：delete_job函数存在底层相关资源没清理干净的问题，后台还占用着这个 job_id，所以再次提交就被拦截了。

用数据库来类别的话，就是：主键字段插了一条数据，表面执行删除了，但实际主键记录没清掉，再插相同主键数据就报错了。

感悟：

第一，前端接口返回成功不代表底层数据和资源真正处理完毕，不能只依靠前端页面提示、接口返回码判断业务实际落地，很多删除接口仅修改逻辑状态，容易忽略缓存、索引、底层资源回收。

第二，做测试要站在全链路视角，不能局限单步功能。删除成功只是中间节点，要验证删除后的衍生场景，也就是 ID 复用、重复新增这类用例。

2, map_groups 设置 concurrency 为负数，任务卡住不退出。
